Adjoint-based Maximisation of Heat Flux in Models of Stellar Convection

Callum Wright-Parish

July 16, 2026

Contents

1 Introduction **1**
1.1 Background 1

2 Problem Setup **2**
2.1 The Adjoint-Based Method 2
2.1.1 Proof of the Adjoint-Based Method 2
2.1.2 The Adjoint-Based Algorithm 4
2.1.3 Advantages over a Brute-Force Method 5
2.2 Examples 5
2.2.1 One-Dimension 5
2.2.2 Two-Dimensions 7
2.3 Rayleigh-Benard Problem 10
2.3.1 Boussinesq Approximation 11
2.3.2 Astrophysical Problem 15

3 Numerical Results **16**
3.1 Domain Width Optimisation 16
3.1.1 Numerical Techniques and Context 16

4 Application to models of Stellar Convection **18**

Abstract

hjfjshfijs

Chapter 1

Introduction

1.1 Background

Chapter 2

Problem Setup

2.1 The Adjoint-Based Method

Before we can construct the problem this paper aims to solve, we need to prove the most general case of an Adjoint-Based Maximisation problem and the algorithm required to solve it.

In this proof, we aim to minimise some objective function $J(\mathbf{y}, \mathbf{u})$ with:

$$\mathbf{y}(\mathbf{x}, t) = (y_1(\mathbf{x}, t), y_2(\mathbf{x}, t), ..., y_m(\mathbf{x}, t))^T \in \mathbb{R}^m$$
$$\mathbf{u}(\mathbf{x}, t) = (u_1(\mathbf{x}, t), u_2(\mathbf{x}, t), ..., u_p(\mathbf{x}, t))^T \in \mathbb{R}^p$$

where $\mathbf{y}$ is our state vector, and $\mathbf{u}$ is our control vector. The state vector, $\mathbf{y}$, has $m-1$ spatial dimensions, and we will choose the control vector, $\mathbf{u}$, to represent the initial conditions of our problem. We have picked this to make the problem setup more intuitive later. $\mathbf{u}(\mathbf{x}, t)$ does not have to be the initial conditions of the system; it can be involved directly in the evolution equation of the system, which we will see in a later example. It can even be a variable defining the size of our domain; however, this does require more work (as we will see).

Essentially, this method will find the control vector $\mathbf{u}$ that minimises (or maximises) some objective function, J, such that $\mathbf{y}$ obeys its evolution equation.

The underlying algorithm we will use to find the optimal $\mathbf{u}$ is a gradient-descent method defined by the following:

$$\mathbf{u}_{n+1} = \mathbf{u}_n - \alpha \nabla_{\mathbf{u}} J \tag{2.1}$$

where $\nabla_{\mathbf{u}} J$ is the total functional derivative (or Fréchet derivative) of J with respect to $\mathbf{u}$, and $\alpha > 0$ is some learning rate. Note that the gradient-descent algorithm minimises J; so if we want to maximise J, we simply minimise $-J$.

2.1.1 Proof of the Adjoint-Based Method

Let our time, $t \in [0, T], T > 0$, and space, $\mathbf{x} \in \Omega \subset \mathbb{R}^{m-1}$, be defined in their respective domains. Let us also define the following inner product:

$$\langle\langle \mathbf{f}, \mathbf{g} \rangle\rangle = \int_0^T \int_\Omega \mathbf{f} \cdot \mathbf{g} d\Omega dt$$

For a general system involving first-order time derivatives, we define the evolution equation and initial conditions as follows:

$$\frac{\partial \mathbf{y}}{\partial t} + \mathcal{A}\mathbf{y} = \mathbf{f}(\mathbf{x}, t) \tag{2.2}$$

$$\mathbf{y}(\mathbf{x}, 0) = \mathcal{B}(u_1(\mathbf{x}), ..., u_p(\mathbf{x}))^T \tag{2.3}$$

where $\mathcal{A}$ is a $m \times m$ linear operator acting on $\mathbf{y}$, $\mathcal{B}$ is an $m \times p$ linear operator, and $\mathbf{u}(\mathbf{x})$ is the initial condition that we want to optimise. It is important to consider some assumptions we now make. We assume that $\mathbf{y}$ is defined on $\Omega \times [0, T]$, and that we can find the respective adjoint operators of $\mathcal{A}$ and $\mathcal{B}$, meaning, for some vector fields $\mathbf{f}$ and $\mathbf{g}$, there exists a linear operator, $\mathcal{A}^*$, such that;

$$\langle\langle \mathbf{g}, \mathcal{A}\mathbf{h} \rangle\rangle = \langle\langle \mathcal{A}^*\mathbf{g}, \mathbf{h} \rangle\rangle \tag{2.4}$$

and similarly, $\mathcal{B}^*$ exists. It is crucial to note that the system of PDE's that we define in (2.2) is linear. If they are not, it requires an extra step to linearise the problem which we will cover when setting up our more complex problem in section 2.3. For now, we will assume that we have a linear system of PDEs.

Now, let us construct some vector field, $\lambda(\mathbf{x}, t) \in \mathbb{R}^m$ and some Lagrangian:

$$\mathcal{L}(\mathbf{y}, \mathbf{u}, \lambda) = J(\mathbf{y}, \mathbf{u}) + \int_0^T \int_\Omega \lambda \cdot \left(\mathbf{f} - \frac{\partial \mathbf{y}}{\partial t} - \mathcal{A}\mathbf{y} \right) d\Omega dt = J(\mathbf{y}, \mathbf{u}) \tag{2.5}$$

Using integration by parts on the second and third term inside the double integral gives the results:

$$\begin{aligned}
\int_0^T \int_\Omega \lambda \cdot (-\mathcal{A}\mathbf{y})\, d\Omega dt &= \langle\langle \lambda, -\mathcal{A}\mathbf{y} \rangle\rangle, \text{then using (2.4)} \\
&= \langle\langle -\mathcal{A}^*\lambda, \mathbf{y} \rangle\rangle \\
&= \int_0^T \int_\Omega (-\mathcal{A}^*\lambda) \cdot \mathbf{y} d\Omega dt \\
\int_0^T \int_\Omega \lambda \cdot \left(-\frac{\partial \mathbf{y}}{\partial t} \right) d\Omega dt &= \int_0^T \int_\Omega \frac{\partial \lambda}{\partial t} \cdot \mathbf{y} d\Omega dt - \int_\Omega \left(\lambda(\mathbf{x}, T) \cdot \mathbf{y}(\mathbf{x}, T) - \lambda(\mathbf{x}, 0) \cdot \mathbf{y}(\mathbf{x}, 0) \right) d\Omega
\end{aligned}$$

Finding this adjoint operator, $\mathcal{A}^*$, will naturally lead to imposing boundary conditions λ must satisfy (see two-dimensional example). We then enforce the initial conditions (2.3) and substitute these results into (2.5), giving the following:

$$\begin{aligned}
\mathcal{L} = J(\mathbf{y}, \mathbf{u}) &+ \int_0^T \int_\Omega \left(\frac{\partial \lambda}{\partial t} - \mathcal{A}^*\lambda \right) \cdot \mathbf{y} d\Omega dt + \int_0^T \int_\Omega \lambda \cdot \mathbf{f} d\Omega dt \\
&- \int_\Omega \left(\lambda(\mathbf{x}, T) \cdot \mathbf{y}(\mathbf{x}, T) - \lambda(\mathbf{x}, 0) \cdot \mathcal{B}\mathbf{u}(\mathbf{x}) \right) d\Omega
\end{aligned}$$

To find a stationary point of J with respect to our state vector, we set the variation $\delta_{\mathbf{y}}\mathcal{L} = 0$. That is, perturbing $\mathbf{y}$ by an infinitesimal amount does not change the value of $\mathcal{L} = J$. For each iteration, $\mathbf{u}_n$ is a fixed vector field - the initial conditions - meaning $\delta\mathbf{y}(\mathbf{x}, 0) = \mathbf{0}$. By taking the variation of $\mathcal{L}$ with respect to $\mathbf{y}$, we get:

$$\begin{aligned}
\delta_{\mathbf{y}}\mathcal{L} &= \mathcal{L}(\mathbf{y} + \delta\mathbf{y}, \mathbf{u}, \lambda) - \mathcal{L}(\mathbf{y}, \mathbf{u}, \lambda) \\
0 = \delta\mathbf{y}\frac{\partial \mathcal{L}}{\partial \mathbf{y}} &\equiv \delta y_i \frac{\partial \mathcal{L}}{\partial y_i}
\end{aligned}$$

We must now assume that our objective function, J, is differentiable across our domain, meaning that it can be written in the form $J(\mathbf{y},\mathbf{u}) = \int_0^T \int_\Omega F(\mathbf{y},\mathbf{u}) d\Omega dt$, which allows us to write the following result:

$$0 = \int_0^T \int_\Omega \left(\partial_{\mathbf{y}} F + \frac{\partial \lambda}{\partial t} - \mathcal{A}^* \lambda \right) \cdot \delta \mathbf{y} d\Omega dt - \int_\Omega \lambda(\mathbf{x}, T) \cdot \delta \mathbf{y}(\mathbf{x}, T) d\Omega$$

We now impose the boundary condition, $\lambda(\mathbf{x}, T) = \mathbf{0}$, and by the Fundamental Lemma of Calculus of Variations, the first integrand must vanish. Hence, in index notation, our vector field, λ, satisfies the following:

$$-\frac{\partial \lambda_i}{\partial t} + \mathcal{A}^*_{ij} \lambda_j = \frac{\partial F}{\partial y_i}, \quad \lambda(\mathbf{x}, T) = 0, \quad i = 1, 2, ..., m \tag{2.6}$$

We now have a system defining what is called the adjoint field, $\lambda(\mathbf{x}, t)$, that satisfies the conditions necessary to find an extremum of our objective function, J. We now need to show how it can actually be used to find the optimal solution. We know the total variation of $\mathcal{L}$ is the sum of its variation arising from $\mathbf{y}$, $\mathbf{u}$ and λ, meaning $\delta\mathcal{L} = \delta_{\mathbf{u}}\mathcal{L} + \delta_{\mathbf{y}}\mathcal{L} + \delta_\lambda \mathcal{L}$. We already imposed $\delta_{\mathbf{y}}\mathcal{L} = 0$, and $\delta_\lambda \mathcal{L} = 0$ is obtained by calculating $\frac{\partial \mathcal{L}}{\partial \lambda}$ from (2.5). Hence, at the extremum, any variation is explained completely by our control vector, $\mathbf{u}$. All this is saying is that J is completely determined by $\mathbf{u}$, which is exactly what we want.

Using these results, we have that $\delta\mathcal{L} = \delta J = \delta_{\mathbf{u}}\mathcal{L}$. Now, from $\mathcal{L}$ we have:

$$\begin{aligned}
\delta_{\mathbf{u}}\mathcal{L} &= \delta_{\mathbf{u}} J + \delta_{\mathbf{u}}(\mathcal{L} - J) \\
\delta J &= \int_\Omega \frac{\partial F}{\partial \mathbf{u}} \cdot \delta \mathbf{u} d\Omega + \int_\Omega \mathcal{B}^* \lambda(\mathbf{x}, 0) \cdot \delta \mathbf{u} d\Omega \\
&= \int_\Omega \left(\frac{\partial F}{\partial \mathbf{u}} + \mathcal{B}^* \lambda(\mathbf{x}, 0) \right) \cdot \delta \mathbf{u} d\Omega
\end{aligned}$$

Note that we only integrate over the spatial domain because $\mathbf{u}$ is strictly defined as an initial condition. By definition, $\delta J = \int_\Omega \nabla_{\mathbf{u}} J \cdot \delta \mathbf{u} d\Omega$, and thus we have the result:

$$\nabla_{\mathbf{u}} J = \frac{\partial F}{\partial \mathbf{u}} + \mathcal{B}^* \lambda(\mathbf{x}, 0) \tag{2.7}$$

We can substitute this into (2.1) to achieve the following form of the gradient-descent algorithm, which will form the backbone for solving our optimisation problems in this paper.

$$\mathbf{u}_{n+1} = \mathbf{u}_n - \alpha \left(\frac{\partial F}{\partial \mathbf{u}} + \mathcal{B}^* \lambda(\mathbf{x}, 0) \right) \tag{2.8}$$

2.1.2 The Adjoint-Based Algorithm

At this point, we have proven that we can evaluate the analytic gradient $\frac{\partial F}{\partial \mathbf{u}} + \mathcal{B}^* \lambda(\mathbf{x}, 0)$ instead of approximating $\nabla_{\mathbf{u}} J$ via finite differences to iterate through control vectors, $\mathbf{u}_n$. Calculating $\frac{\partial F}{\partial \mathbf{u}}$ is far easier than calculating a total derivative, as it is usually something we can do analytically. This is because we do not need to calculate $\frac{\partial J}{\partial \mathbf{x}} \frac{d\mathbf{x}}{d\mathbf{u}}$ across our domain, which would be computationally expensive.

To carry out the adjoint-based maximisation algorithm for the general use-case of $\mathbf{u}_n$, we need to carry out the following steps:

1. Start with an initial guess, $\mathbf{u}_0(\mathbf{x})$, for the optimal initial conditions.
2. Forward solve the system (2.2) using $\mathbf{u}_n(\mathbf{x})$ to define an updated problem, then find $\mathbf{y}_n(\mathbf{x}, t)$.
3. Evaluate $J(\mathbf{y}_n, \mathbf{u}_n)$. If $|J(\mathbf{y}_n, \mathbf{u}_n) - J(\mathbf{y}_{n-1}, \mathbf{u}_{n-1})| < \epsilon$, some tolerance, then stop.
4. Otherwise, solve the system (2.6) going backwards in time (starting with $t = T$) to find the adjoint field $\lambda(\mathbf{x}, t)$. In practice, we let $\tau = T - t$ and solve for $\tau \in [0, T]$.
5. We then use the adjoint field's 'final' state of $\lambda(\mathbf{x}, t = 0)$ to update $\mathbf{u}_k$.
6. Repeat steps 2-5 until J converges, or for N iterations so $n = 1, 2, ..., N$.

2.1.3 Advantages over a Brute-Force Method

Using the adjoint-based method on a discretised domain to extremise an objective function is computationally viable compared to a brute-force method. For each iteration of $\mathbf{u}_n$ with the adjoint-based method, we require one forward pass and one backward pass; we only have to solve two PDE's over our time domain. Whereas using a brute-force method requires that we complete a forward pass for every (discrete) point on the control function to calculate the gradient approximation $\frac{dJ}{du_i}$ on each iteration of $\mathbf{u}_n$. In large spatial domains, a brute-force method is almost computationally impossible due to processing and storage constraints.

So far, this may seem rather abstract and theoretical, so I have worked through the previous proof for a simple one-dimensional example below, where $\mathbf{u}$ does not represent the initial conditions, showing the generality of this method. It also outlines the benefits of using this adjoint-based method over a brute-force method. I will then demonstrate a second example in higher dimensions, using $\mathbf{u}(\mathbf{x})$ to represent the system's initial conditions.

2.2 Examples

2.2.1 One-Dimension

This basic example does not include a time derivative term (the next example does), but still employs the adjoint-based optimisation method to minimise an objective function, $J(y, u)$, given some system defined by some linear operator and boundary conditions. Essentially, this solution finds the optimal function, $u(x)$, such that $y(x)$ satisfies the ODE and is as 'close' to $y_{target}(x)$ as possible. To avoid loss of generality, we define the system as follows.

$$Ly \equiv \frac{dy}{dx} + ay = u(x), \quad y(0) = 0, \quad x \in [0, L], \quad a \in \mathbb{R} \tag{2.9}$$

$$J = \frac{1}{2}\int_0^L \left(y(x) - y_{target}(x)\right)^2 dx \tag{2.10}$$

Note that $\mathcal{A} = a$ in this example, which is self-adjoint (so $\mathcal{A}^* = a$). We want to minimise $J(y, u)$, so we will be using the Gradient-Descent algorithm to iterate the function u, N times. That is,

$$u_{n+1} = u_n - \alpha \nabla_u J$$

for $n = 0, 1, ..., N$, where u_0 is some initial guess, and α is some constant learning rate. Firstly, it is important to note that by definition:

$$\delta_u J = \int_0^L \nabla_u J(x) \delta u(x) dx$$

Now construct the Lagrangian, $\mathcal{L} = J$, for which we want its variation to be zero, and some function $\lambda(x)$.

$$\mathcal{L} = \frac{1}{2} \int_0^L (y - y_{target})^2 dx + \int_0^L \lambda(x) \left(u(x) - \frac{dy}{dx} - ay \right) dx = J$$

Using integration by parts: $\int_0^L \lambda(x) \frac{dy}{dx} dx = [\lambda(x) y(x)]_0^L - \int_0^L y \frac{d\lambda}{dx} dx$

$$\implies \mathcal{L} = \int_0^L \left(\frac{1}{2} (y - y_{target})^2 + y \left(\frac{d\lambda}{dx} - a\lambda \right) + \lambda u \right) dx - \lambda(L) y(L) + \lambda(0) y(0)$$

Now, impose $\lambda(L) = 0$, and set the variation with respect to y, $\delta_y \mathcal{L} = 0$ for some arbitrary δy:

$$\begin{aligned}
\delta_y \mathcal{L} &= \delta y \frac{d\mathcal{L}}{dy} + \mathcal{O}((\delta y)^2) = 0 \\
&\implies \delta y \frac{d\mathcal{L}}{dy} = 0 = \delta y \frac{d}{dy} \int_0^L \left(\frac{1}{2} (y - y_{target})^2 + y \left(\frac{d\lambda}{dx} - a\lambda \right) + \lambda u \right) dx \\
0 &= \delta y \int_0^L \frac{\partial}{\partial y} \left(\frac{1}{2} (y - y_{target})^2 + y \left(\frac{d\lambda}{dx} - a\lambda \right) + \lambda u \right) dx \\
&\implies 0 = \int_0^L \left(y - y_{target} + \frac{d\lambda}{dx} - a\lambda \right) \delta y dx
\end{aligned}$$

By the Fundamental Lemma of Calculus of Variations, the integrand must vanish:

$$\implies -\frac{d\lambda}{dx} + a\lambda = y - y_{target}, \quad \lambda(L) = 0 \tag{2.11}$$

Note that in this example, $F = \frac{1}{2}(y - y_{target})^2$, which now appears in (2.11) via $\frac{\partial F}{\partial y}$. This evolution equation for λ is in the form seen in (2.6). This also means that we would expect only a λ-term to appear in the $\nabla_u J$ term as $\frac{\partial J}{\partial u} = 0$. Now, let us focus on the variation in the u-component to first order δu:

$$\begin{aligned}
\delta_u \mathcal{L} &= \delta u \frac{d\mathcal{L}}{du} + \mathcal{O}((\delta u)^2) = \delta_u J \\
&= \delta u \frac{d}{du} \int_0^L \left(\frac{1}{2} (y - y_{target})^2 + y \left(\frac{d\lambda}{dx} - a\lambda \right) + \lambda u \right) dx \\
&= \delta u \int_0^L \frac{\partial}{\partial u} \left(\frac{1}{2} (y - y_{target})^2 + y \left(\frac{d\lambda}{dx} - a\lambda \right) + \lambda u \right) dx \\
\delta_u J &= \int_0^L \lambda(x) \delta u(x) dx = \int_0^L \nabla_u J \delta u(x) dx \\
&\implies \nabla_u J = \lambda(x)
\end{aligned}$$

This is in the form we expected. Hence, we can use $\lambda(x)$ instead of $\nabla_u J$ when iterating u_n, which is advantageous because solving for $\nabla_u J$ via brute-force would be computationally expensive. Our discretised domain is formed of k x-values, x_i, $i = 1, 2, ..., k$. On each iteration of u_n, the brute-force method requires:

1. For the point on the grid $u_n(x_i) = u_i$, calculate an initial value $J_{n,i}$.
2. Perturb the point on the grid by a small amount, ϵ.
3. Calculate a new value $J_{n+1,i}$.
4. Calculate $\frac{\partial J}{\partial u_i} \approx \frac{J_{n+1} - J_n}{\epsilon}$.
5. Return u_i to where it was, and repeat 1-4 for every u_i.
6. Transform each point $u_i \to u_i - \alpha \frac{\partial J}{\partial u_i} \approx u_i - \alpha \frac{J_{n+1,i} - J_{n,i}}{\epsilon}$ to give the points on u_{n+1}.

Thus, each iteration of u_n would otherwise require solving for y (a forward pass), k times. Whereas, if we use the adjoint-based method, we do one forward pass and then one backward pass (solve for λ where λ satisfies (2.11)) per iteration of u_n. That is, the brute-force method would require kN passes, whereas using the adjoint-based method simply requires $2N$ passes. Therefore, we use the iterative method:

$$u_{n+1}(x) = u_n(x) - \alpha\lambda(x)$$

2.2.2 Two-Dimensions

In this example, I will focus less on the rigorous proof behind the theory and more on the computational practice of finding adjoint operators and deriving appropriate boundary conditions for the adjoint field, λ. We will define a two-dimensional (one spatial and one temporal) problem describing heat diffusivity.

Let us have a wall between two regions of fixed temperature, and we want to maximise the heat flux through the wall. We define our state vector (scalar temperature in this case), $\mathbf{y} = y(x, t)$, and control vector, $\mathbf{u} = u(x)$, where $x \in [0, L]$ and $t \in [0, T]$. Of course, using three spatial dimensions would lead to a more comprehensive understanding of this problem, but for simplicity, we will use one spatial dimension for now. In this example, we define the system as follows:

$$\begin{aligned} \frac{\partial y}{\partial t} - \nu \frac{\partial^2 y}{\partial x^2} &= 0 \\ y(x, 0) &= u(x) \\ y(L, t) = 0, \quad y(0, t) &= 100 \end{aligned}$$

with our objective function including the heat flux, and some weighted term to stop the optimiser setting $u(x, t) = \pm\infty$.

$$J = \int_0^T \int_0^L \left(-\nu \frac{\partial y}{\partial x} + \frac{\gamma}{2} (u(x))^2 \right) dx dt$$

Notice this is in the form of $J(\mathbf{y}, \mathbf{u}) = \int_0^T \int_\Omega F(\mathbf{y}, \mathbf{u}) d\Omega dt$. We can also notice that the linear operator, $\mathcal{A} = -\frac{\partial^2}{\partial x^2}$, and that $\mathcal{B} = 1$ in this case. Both are trivially self-adjoint, but we will need to formally show that $\mathcal{A}$ is self-adjoint to work out the appropriate boundary conditions needed to solve for $\lambda(x, t)$.

As we have done in the proof and previous example, let

$$\mathcal{L} = J(y,u) + \int_0^T \int_0^L \lambda \left(-\frac{\partial y}{\partial t} + \frac{\partial^2 y}{\partial x^2} \right) dxdt \tag{2.12}$$

Now using integration by parts:

$$\begin{aligned}\int_0^T \int_0^L \lambda \frac{\partial^2 y}{\partial x^2} dxdt &= \int_0^T \left[\lambda \frac{\partial y}{\partial x} \right]_0^T dt - \int_0^T \int_0^L \frac{\partial \lambda}{\partial x} \frac{\partial y}{\partial x} dxdt \\ &= \int_0^T \left[\lambda \frac{\partial y}{\partial x} - \frac{\partial \lambda}{\partial x} y \right]_0^L dt + \int_0^T \int_0^L y \frac{\partial^2 \lambda}{\partial x^2} dxdt\end{aligned}$$

This verifies that $\mathcal{A}$ is, in fact, self-adjoint subject to λ fulfilling some boundary conditions. We need

$$\left[\lambda \frac{\partial y}{\partial x} - \frac{\partial \lambda}{\partial x} y \right]_0^L = 0$$

We do not have homogeneous Dirichlet or Neumann boundary conditions, meaning we must use Calculus of Variations to derive the boundary conditions for λ. We mentioned in the proof that at the boundary conditions of our state vector, its variation is zero. That is

$$y(0,t) = 100 \implies \delta y(0,t) = 0 \quad \text{and} \quad y(L,t) = 0 \implies \delta y(L,t) = 0$$

The equations defining this system must also hold if we were to perturb y some small amount, namely, they must hold for $y^* = y + \delta y$. As derivatives are associative, we know that δy must also satisfy our system of equations. This means that we can have

$$\begin{aligned}\left[\lambda \frac{\partial \delta y}{\partial x} - \frac{\partial \lambda}{\partial x} \delta y \right]_0^L &= 0 \\ \lambda(L,t) \frac{\partial(\delta y)}{\partial x}(L,t) - \frac{\partial \lambda}{\partial x}(L,t) \delta y(L,t) - \lambda(0,t) \frac{\partial(\delta y)}{\partial x}(0,t) + \frac{\partial \lambda}{\partial x}(0,t) \delta y(0,t) &= 0 \\ \lambda(L,t) \frac{\partial(\delta y)}{\partial x}(L,t) - \lambda(0,t) \frac{\partial(\delta y)}{\partial x}(0,t) &= 0\end{aligned}$$

Now, we can impose the boundary conditions $\lambda(L,t) = \lambda(0,t) = 0$ necessary for $\mathcal{A}$ to have an adjoint operator. From the other term in the integral in (2.12), we have that

$$\begin{aligned}\int_0^T \int_0^L -\lambda \frac{\partial y}{\partial t} dxdt &= \int_0^T \int_0^L \frac{\partial \lambda}{\partial t} y dxdt - \int_0^L (\lambda(x,T) y(x,T) - \lambda(x,0) y(x,0)) dx \\ \implies \mathcal{L} &= J(y,u) + \int_0^T \int_0^L \left(\frac{\partial \lambda}{\partial t} - \frac{\partial^2 \lambda}{\partial x^2} \right) y dxdt \\ &\quad - \int_0^L (\lambda(x,T) y(x,T) - \lambda(x,0) y(x,0)) dx\end{aligned}$$

We impose the 'initial' condition $\lambda(x,T) = 0$ and we know

$$\begin{aligned}F &= -\nu \frac{\partial y}{\partial x} + \frac{\gamma}{2} (u(x))^2 \\ \implies \frac{\partial F}{\partial y} &= -\nu \frac{\partial}{\partial y} \left(\frac{\partial y}{\partial x} \right) = -\nu \frac{\partial}{\partial x} \left(\frac{\partial y}{\partial y} \right) = -\nu \frac{\partial}{\partial x}(1) = 0\end{aligned}$$

Hence, using (2.6), we find that λ satisfies the system:

$$\begin{aligned} -\frac{\partial \lambda}{\partial t} + \frac{\partial^2 \lambda}{\partial x^2} &= 0 \\ \lambda(x, T) &= 0 \\ \lambda(L, t) = \lambda(0, t) &= 0 \end{aligned}$$

This system can be numerically (and analytically) solved. Solving for $\nabla_u J$ now becomes a simple task as a constant linear operator, $\mathcal{B}$, is trivially self-adjoint. We can now return to the original problem of minimising our objective function, J. From (2.8), we can iterate u_n via the iterative formula $u_{n+1}(x) = u_n(x) - \alpha(\gamma u_n(x) + \lambda(x, 0))$

$$\implies u_{n+1}(x) = (1 - \alpha\gamma)u_n(x) - \alpha\lambda(x, 0)$$

It may seem counter-intuitive to say the 'initial' condition of the adjoint field is at $t = T$, but this is because we are solving for λ going backwards in time - starting at $t = T$, and solving for the function $\lambda(x, 0)$.

We have now shown how we can optimise the initial conditions of a two-dimensional problem to maximise some arbitrary objective function. When solving this problem numerically, setting $\alpha = 0.05$, $\gamma = 1$, $\nu = 0.1$, and $u_0(x) = 100 - 50x$ we achieve the following results.

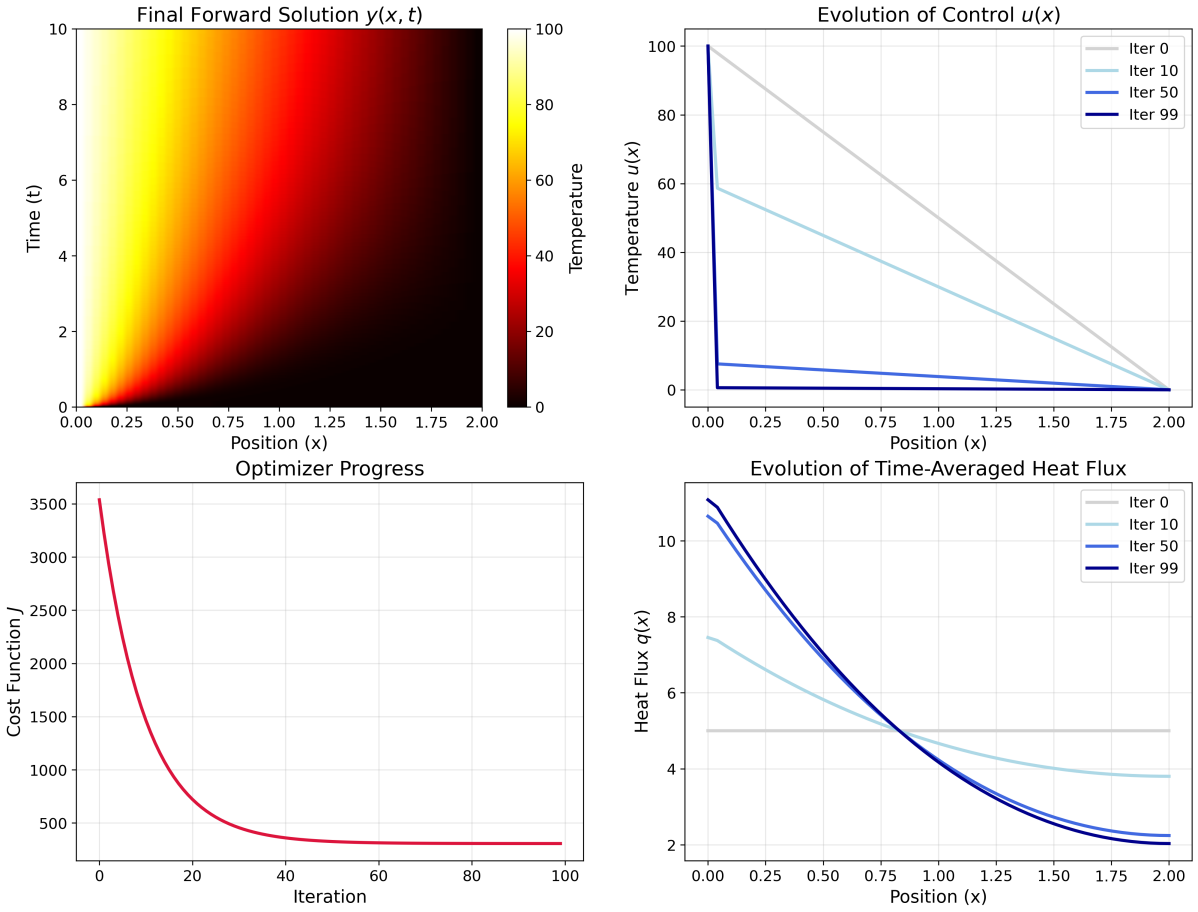


Figure 2.1: Two-Dimensional Adjoint Optimisation Results

Figure 2.1 shows that the optimal initial conditions to maximise the total heat flux = (integral under the time-averaged heat flux graph)$\times T$, across our domain tend towards $u(x) = 0, x \in (0, L]$ and $u(0) = 100$. To verify this is the case, when we run the optimisation setting this as the initial conditions, J becomes constant. This makes sense mathematically as it creates a step at $x = 0$, giving an effectively infinite flux at this point. Of course, setting $u_0(x) = 100, x \in [0, L)$ and $u_0(L) = 0$, would also created a step at $x = L$. However, the weighted $(u(x))^2$ term ensures that u is made as small as possible. In fact, the final initial condition this optimisation created, $u_{100}(x)$, is the best solution possible given the step size in x, δx. Choosing a smaller δx would result in initial conditions closer to the true 'best' initial conditions.

2.3 Rayleigh-Benard Problem

We have proven the adjoint-based method is a viable method for optimising PDE problems, and gone through some examples; we can now begin to add complexity to construct the main problems this paper aims to solve.

We will be focusing on using the Navier-Stokes equations to model fluids within various geometries throughout these problems, starting with the incompressible case. However, we will first cover the general compressible Navier-Stokes equations and then impose constraints suitable to our problems. In the most general sense, the Navier-Stokes equations (in 3-dimensions) are simply an application of Newton's Second Law, and other fundamental conservation laws. We define the fluid density $\rho(\mathbf{x}, t)$, fluid velocity $\mathbf{v}(\mathbf{x}, t)$, temperature $T(\mathbf{x}, t)$, pressure $p(\mathbf{x}, t)$, and $\mathbf{F}$ representing external forces, giving the following equations in Cartesian coordinates:

$$\frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \mathbf{v}) = 0 \quad \text{(Conservation of Mass)} \tag{2.13}$$

$$\rho \left(\frac{\partial \mathbf{v}}{\partial t} + \mathbf{v} \cdot \nabla \mathbf{v} \right) = -\nabla p + \nabla \cdot \tau + \mathbf{F} \quad \text{(Newton's Second Law)} \tag{2.14}$$

$$\rho c_v \left(\frac{\partial T}{\partial t} + \mathbf{v} \cdot \nabla T \right) = -p \left(\nabla \cdot \mathbf{v} \right) + k \nabla^2 T + tr(\tau \nabla \mathbf{v}) \quad \text{(Advection-Diffusion Equation)} \tag{2.15}$$

where

$$\nabla \mathbf{v} = \begin{pmatrix} \frac{\partial u}{\partial x} & \frac{\partial u}{\partial y} & \frac{\partial u}{\partial z} \\ \frac{\partial v}{\partial x} & \frac{\partial v}{\partial y} & \frac{\partial v}{\partial z} \\ \frac{\partial w}{\partial x} & \frac{\partial w}{\partial y} & \frac{\partial w}{\partial z} \end{pmatrix}, \quad \tau = \mu \left(\nabla \mathbf{v} + (\nabla \mathbf{v})^T - \frac{2}{3} (\nabla \cdot \mathbf{v}) I \right)$$

These equations are well-known and well-studied [5], but very complicated to solve numerically, and often impossible to solve analytically. They define the constants for dynamic viscosity μ, thermal conductivity k, and specific heat capacity at a fixed volume, c_v.

Throughout these problems, we will also focus heavily on the Nusselt Number, Nu, defined as the ratio of heat transported by convection to that transported by conduction. It is an important quantity in gaining an understanding of how heat is transported across a layer of fluid, and will be a quantity we aim to maximise in our problems.

In our first problem, we will apply a widely used approximation to the Navier-Stokes equations, and then build on this in the following problems.

2.3.1 Boussinesq Approximation

Equations and Domain

In our first problem, we will use the widely accepted Boussinesq approximation to the compressible Navier-Stokes equations [3]. It treats changes in the fluid's density as effectively negligible - essentially constant density. That is,

$$\frac{\partial \rho}{\partial t} = 0 \implies \nabla \cdot \mathbf{v} = 0 \quad \text{by (2.13)}$$

This then makes the rest of our equations much more manageable.

$$\frac{\partial \mathbf{v}}{\partial t} + \mathbf{v} \cdot \nabla \mathbf{v} = -\frac{1}{\rho}\nabla p + \frac{\mu}{\rho}\nabla^2 \mathbf{v} + \frac{1}{\rho}\mathbf{F} \tag{2.16}$$

$$\frac{\partial T}{\partial t} + \mathbf{v} \cdot \nabla T = \kappa \nabla^2 T + \Phi \tag{2.17}$$

where Φ is some internal heating, and κ is the thermal diffusivity. In our problem, we will be setting $\Phi = 0$, as we focus on heating via the boundaries. We are now considering the incompressible Navier-Stokes equations, $\rho = \rho_0$.

To reduce the number of constants we're dealing with, we non-dimensionalise our equations by scaling each variable. We scale distance by the height of the layer of fluid, H, time by the thermal diffusion time, H^2/κ, and pressure by $\rho_0 H^2/\kappa$, as demonstrated in Goluskin, 2015 [3]. We then also use an approximation to the force created through buoyancy,

$$\frac{1}{\rho_0}\mathbf{F} = -g(1 - \beta(T - T_0))\hat{\mathbf{e}}_z$$

where T_0 is the temperature of the top boundary, and β is the coefficient of thermal expansion. When we apply these to the above equations, we get the non-dimensionalised Boussinesq approximation equations:

$$\nabla \cdot \mathbf{v} = 0 \tag{2.18}$$

$$\frac{\partial \mathbf{v}}{\partial t} + \mathbf{v} \cdot \nabla \mathbf{v} = -\nabla p + Pr\nabla^2 \mathbf{v} + PrRaT\hat{\mathbf{e}}_z \tag{2.19}$$

$$\frac{\partial T}{\partial t} + \mathbf{v} \cdot \nabla T = \nabla^2 T + Q \tag{2.20}$$

where Q is some internal heating that we will set to zero (but kept for now for the sake of generality). We also have defined some well-known constants, the Rayleigh and Prandtl numbers:

$$Ra := \frac{g\beta H^3 \Delta T}{\kappa \nu}$$

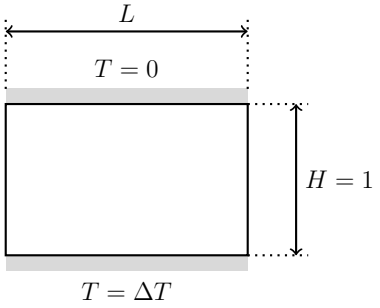
$$Pr := \frac{\kappa}{\nu}$$

where H is the height of our layer of fluid, ν is kinematic viscosity, and ΔT is the difference in temperature between the two layers. The Rayleigh number is indicative as a ratio of inertial forces to viscous forces - a large Rayleigh number indicates that the fluid is being largely influenced by buoyancy forces, $\mathbf{F}$. There is a minimum Rayleigh number required for convection to occur for different boundary conditions. [7][4][2][6]

No-slip boundaries	$\mathbf{v}(x,y,0,t) = \mathbf{v}(x,y,H,t) = \mathbf{0}$	$R_{min} \approx 1707.96$
Free-slip boundaries	$v_z(x,y,0,t) = v_z(x,y,H,t) = 0$	$R_{min} \approx 657.511$
Mixed (one free-slip, one no-slip)	$\mathbf{v}(x,y,0,t) = \mathbf{0}, v_z(x,y,H,t) = 0$	$R_{min} \approx 1100.65$

The Prandtl number is a material property of each fluid, and it a measure of how quickly a fluid diffuses momentum compared to how fast it diffuses heat. A high Prandtl number represents a fluid that largely damps motion and is a poor conductor of heat, such as in the Earth's mantle. A small Prandtl number represents a fluid that weakly damps motion and conducts heat well, such as stellar planets.

Now we have built some intuition of the model defined in equations (2.18)-(2.20), we can begin to define the geometry for this problem. We will be focusing on a two-dimensional (x, z), problem modelling a layer of fluid in a domain of height $H = 1$ and width L. The fluid will be heated from the bottom, and cooled at the top.



Adjoint-Based Method Application

From here, we can begin to explore how we can apply the adjoint-based method of optimisation to this setup. We previously defined the Nusselt number qualitatively as the ratio of heat transported via convection as opposed to that transferred by conduction. In this problem, we are interested in the optimal value for L that will maximise this Nusselt number.

We will solve this numerically through simulation across time $t \in [0, t_*]$. The Nusselt number is calculated at an instantaneous moment in time by

$$Nu = 1 + \langle v_z T \rangle$$

where v_z is the vertical component of a particles velocity, and $\langle \cdot \rangle$ represents an average across the domain [3]. To maximise the time-averaged Nusselt number, and have an objective function in the form we want, we define it as follows:

$$J = \int_0^{t_*} 1 + \frac{H}{\beta |\Omega| \Delta T} \int_\Omega v_z T \, d\Omega dt$$
$$\implies J = \int_0^{t_*} \int_\Omega \frac{1}{|\Omega|} \left(1 + \frac{1}{\beta \Delta T} v_z T \right) d\Omega dt$$

where β is our coefficient of thermal expansion, and $|\Omega|$ is the unit area/volume of the spatial domain. Through the construction of this problem, we get the final form of the objective function to be:

$$J = \int_0^{t_*} \int_0^1 \int_0^L \frac{1}{L} \left(1 + \frac{1}{\beta \Delta T} v_z T \right) dx dz dt \tag{2.21}$$

So far, we have defined the system of equations defining our system, as well as the objective function to maximise. Next, we will work out how we can apply the adjoint-based method to optimise the domain width, L, to maximise J. Before we can find the adjoint equations, we will see no-slip boundary conditions, which are representative of

many laboratory and engineering applications. We will use free-slip boundary conditions later when considering astrophysical flows. For now, we have set $\mathbf{v}(x,0,t) = \mathbf{v}(x,H,t) = \mathbf{0}$. Now we can find the adjoint equations required to solve this problem.

Linearisation

The first hurdle to overcome in constructing this adjoint field is that not all the terms in (2.19) and (2.20) are linear. Therefore, we must linearise each state about its 'equilibrium state', assuming that all terms of quadratic order or higher are negligible when calculating the effect of small perturbations. In practise, this is okay for accuracy as long as we set the discretised time step, δt, to be sufficiently small. Essentially, this offloads some analytical complexity onto the numerical solver. In this problem, our state vector $\mathbf{y} = (\mathbf{v}, T, p)^T$ (and control vector $\mathbf{u} = (L)$), with $\rho = \rho_0$, so we define the linearising transformation:

$$\mathbf{y} = \bar{\mathbf{y}} + \mathbf{y}'$$

This causes our incompressible Boussinesq approximation equations to take a linear form:

$$\nabla \cdot \mathbf{v}' = 0 \tag{2.22}$$

$$\frac{\partial \mathbf{v}'}{\partial t} + \bar{\mathbf{v}} \cdot \nabla \mathbf{v}' + \mathbf{v}' \cdot \nabla \bar{\mathbf{v}} = -\nabla p' + Pr\nabla^2 \mathbf{v}' + PrRaT'\hat{\mathbf{e}}_z \tag{2.23}$$

$$\frac{\partial T'}{\partial t} + \bar{\mathbf{v}} \cdot \nabla T' + \mathbf{v}' \cdot \nabla \bar{T} = \nabla^2 T' \tag{2.24}$$

In our simulations, we will use the non-linear equations, but to analytically find the adjoint field, we will use this linear form. We now follow a similar procedure as in the examples, and the proof (albeit more complicated) to find the adjoint problem. Let us define the Lagrangian,

$$\begin{aligned}\mathcal{L} = J + \int_0^{t_*}\int_\Omega \lambda \cdot \left(-\frac{\partial \mathbf{v}'}{\partial t} - \bar{\mathbf{v}} \cdot \nabla \mathbf{v}' - \mathbf{v}' \cdot \nabla \bar{\mathbf{v}} - \nabla p' + Pr\nabla^2 \mathbf{v}' + PrRaT'\hat{\mathbf{e}}_z\right) d\Omega dt \\ + \int_0^{t_*}\int_\Omega \pi(-\nabla \cdot \mathbf{v}') d\Omega dt + \int_0^{t^*}\int_\Omega \theta\left(-\frac{\partial T'}{\partial t} - \bar{\mathbf{v}} \cdot \nabla T' - \mathbf{v}' \cdot \nabla \bar{T} + \nabla^2 T'\right) d\Omega dt\end{aligned}$$

where $(\lambda, \pi, \theta)^T$ is the adjoint field. We use integration by parts and apply the boundary conditions to each term in these integrals to rearrange the terms. We then take advantage of the form $J = \int_0^{t^*}\int_\Omega F d\Omega dt$, and then take the variation of $\mathcal{L}$ with respect to $\mathbf{v}'$, p' and T' respectively to achieve the following adjoint problem.

$$-\frac{\partial \lambda}{\partial t} - \bar{\mathbf{v}} \cdot \nabla \lambda + (\nabla \bar{\mathbf{v}})^T \lambda = \nabla \pi + Pr\nabla^2 \lambda - \theta \nabla \bar{T} + \frac{\partial F}{\partial \mathbf{v}'} \tag{2.25}$$

$$\nabla \cdot \lambda = -\frac{\partial F}{\partial p'} \tag{2.26}$$

$$-\frac{\partial \theta}{\partial t} - \bar{\mathbf{v}} \cdot \nabla \theta = \nabla^2 \theta + PrRa\lambda_z + \frac{\partial F}{\partial T'} \tag{2.27}$$

where we set $\lambda(\mathbf{x}, t^*) = \mathbf{0}$, $\lambda(x,0,t) = \lambda(x,H,t) = \mathbf{0}$, $\theta(\mathbf{x}, t^*) = 0$, and $\theta(x,0,t) = \theta(x,H,t) = 0$. Once we solve this system, going backwards in time, the adjoint field is used to update domain width, L. However, this is where we encounter the next major hurdle in this method.

Re-scaling

Since our objective function depends on our control vector within the bounds of its integral, we cannot directly use the form of the gradient-descent method in the proof. However, the workaround is only a few extra steps. We want to remove this dependence on L in the limits of the integral, meaning we need to scale our x-coordinates, letting $\hat{x} = x/L$. This leads to the transformed objective function,

$$J = \int_0^{t^*} \int_0^1 \int_0^1 \left(1 + \frac{1}{\beta \Delta T} v_z T\right) d\hat{x} dz dt$$

It also means that we must return to equations (2.18)-(2.20) and (2.22)-(2.27) to transform the x-coordinates throughout the equations. To do this, we define the linear operators

$$\nabla_L = \left(\frac{1}{L}\frac{\partial}{\partial x}, \frac{\partial}{\partial z}\right), \quad \nabla_L^2 = \frac{1}{L^2}\frac{\partial^2}{\partial x^2} + \frac{\partial^2}{\partial z^2}$$

and apply them to these equations. Thus, we have the final, non-linear, transformed forward system to be

$$\nabla_L \cdot \mathbf{v} = 0 \tag{2.28}$$

$$\frac{\partial \mathbf{v}}{\partial t} + \mathbf{v} \cdot \nabla_L \mathbf{v} = -\nabla_L p + Pr \nabla_L^2 \mathbf{v} + PrRaT\hat{\mathbf{e}}_z \tag{2.29}$$

$$\frac{\partial T}{\partial t} + \mathbf{v} \cdot \nabla_L T = \nabla_L^2 T \tag{2.30}$$

and after linearising each term through a similar method as above, we reach the final transformed, linear adjoint problem

$$-\frac{\partial \lambda}{\partial t} - \bar{\mathbf{v}} \cdot \nabla_L \lambda + (\nabla_L \bar{\mathbf{v}})^T \lambda = \nabla_L \pi + Pr \nabla_L^2 \lambda - \theta \nabla_L \bar{T} + \frac{\bar{T}}{\beta \Delta T} \hat{\mathbf{e}}_z \tag{2.31}$$

$$\nabla_L \cdot \lambda = 0 \tag{2.32}$$

$$-\frac{\partial \theta}{\partial t} - \bar{\mathbf{v}} \cdot \nabla_L \theta = \nabla_L^2 \theta + PrRa\lambda_z + \frac{\bar{v_z}}{\beta \Delta T} \tag{2.33}$$

subject to the same initial conditions as previously defined. We have successfully transformed our objective function into the appropriate form by transforming the entire problem to shift the dependency of L into the equations themselves. However, in fixing one problem, we have created another. Due to the way we transformed x, we removed any dependency on L inside the integral, meaning we cannot easily calculate $\frac{dJ}{dL}$. By taking the derivative $\frac{d\mathcal{L}}{dL}$, and applying the chain rule, we find

$$\frac{dJ}{dL} = \int_0^{t_*} \int_0^1 \int_0^1 \left[\frac{\pi}{L}\frac{\partial v_x}{\partial \hat{x}} + \lambda \cdot \left(\frac{v_x}{L}\frac{\partial \mathbf{v}}{\partial \hat{x}} + \frac{1}{L}\frac{\partial p}{\partial \hat{x}} - \frac{2Pr}{L^2}\frac{\partial^2 \mathbf{v}}{\partial \hat{x}^2}\right) + \theta\left(\frac{v_x}{L}\frac{\partial T}{\partial \hat{x}} - \frac{2}{L^2}\frac{\partial^2 T}{\partial \hat{x}^2}\right)\right] d\hat{x} dz dt$$

This is not a pleasant integral to solve analytically, so we will solve it numerically, which is more manageable. The main drawback with this form is that we have to store the historic states on the adjoint field throughout the backward pass to calculate the complete time integral. Therefore, computationally, we may face bottlenecks depending on storage constraints. To alleviate this constraint, we could relax the problem to focus on

maximising Nu at the end of our simulation, $t = t^*$, thereby removing the time integral. We will look at both this relaxed case, as well as the more computationally intense case. This is still far more efficient than brute force in this case, and easily scalable to use more control variables.

Therefore, we will numerically solve the forward problem defined in equations (2.28)-(2.30), then solve the backward problem (equations (2.31)-(2.33)). Calculate the volume integral defined about, then iterate through values of L using

$$L_{n+1} = L_n + \alpha \frac{dJ}{dL}$$

The code used to carry out these simulations can be found in my GitHub repository. The code for this problem can be found in the `../Adjoint_Problems/` folder.

2.3.2 Astrophysical Problem

Chapter 3

Numerical Results

We have now completed the appropriate theoretical setup for each problem. This section will present our numerical results found through various simulation techniques.

3.1 Domain Width Optimisation

In this problem, we aim to optimise the width, L, of our rectangular domain to maximise the time-averaged Nusselt number. We use the adjoint-based maximisation method to efficiently iterate through different values of L, eventually converging to a final optimal value. We will investigate our simulation findings and discuss whether our numerical results align with the mathematical theory.

So far, we have focused on the mathematical generality of the adjoint-based method for optimising a certain quantity associated with any system of differential equations. However, to fully understand these results, we need to understand some of the numerical techniques used to solve equations (2.28)-(2.33).

3.1.1 Numerical Techniques and Context

We are using the Dedalus Project framework for solving partial differential equations using spectral methods. When defining our problem in Python, we are required to specify the basis for each coordinate. For the z-coordinate, we choose a `ChebyshevT` basis, as this concentrates more points towards the boundaries of its domain. This is a good choice for our problem due to the extreme boundary conditions. For the x-coordinate, we choose a `RealFourier` basis, meaning numerical solutions in the x-direction will be combinations of sine and cosine waves. A term in these series solutions is referred to as a 'node', and each of the $\hat{N}$ nodes has its own associated wavenumber $k_n = \frac{2\pi n}{L}$, $n = 0, 1, 2, ..., \hat{N}$.

From mathematical theory [3][7][6][2][4], at a given Rayleigh number, a fluid has a natural wavenumber (much like a tuning fork has a natural frequency) at which it transports the most heat by convection. Therefore, if one of our n nodes has a wavenumber similar to (or the same as) this natural wavenumber, we would expect it to be responsible for most of the heat transport in the system. Since we can change the wavenumbers of our modes by changing L, finding the optimal value of L for transporting heat via convection would be analogous to finding the series solution that contains the natural wavenumber at the given Rayleigh number. It then requires a decomposition of this series solution to

investigate which node in the solution is responsible for most of the heat transport.

Finding this natural wavenumber is important for deducing fluid properties, but could also allow for simpler models of fluids with little loss of detail.

When running this code, we have to find a balance between speed and accuracy. In the construction of our domain, we discretise the domain into $N_x \times N_z$ individual cells. The larger we make this domain, the more accuracy we get, but the longer it takes to compute each iteration of L_n. Depending on available hardware, the optimal domain discretisation varies [1]- this is beyond the scope of this paper.

Chapter 4

Application to models of Stellar Convection

Bibliography

[1] Feeney A. Li Z. Bostanabad R. Chandramowlishwaran A. Breaking boundaries: Distributed domain decomposition with scalable physics-informed neural pde solvers. Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, 1–15, 2023.

[2] Low A.R. On the criterion for stability of a layer of viscous fluid heated from below. Proc. R. Soc. A, 1929.

[3] Goluskin D. Internally heated convection and rayleigh-bénard convection. physics.flu-dyn, 2015.

[4] Jeffreys H. Some cases of instability in fluid motion. Proc. R. Soc. A, 1928.

[5] navier stokes.org. Deriving the navier-stokes equations. navier-stokes.org, 2026.

[6] Southwell R.V. Pellew A. On maintained convective motion in a fluid heated from below. Proc. R. Soc. A, 1940.

[7] Lord Rayleigh. On convection currents in a horizontal layer of fluid, when the higher temperature is on the under side. Phil. Mag., 1916.